

Лабораторная работа №14

Программирование в командном процессоре ОС UNIX

Липатников Андрей

2026.10.08

В данном отчёте представлены результаты выполнения лабораторной работы по теме «Программирование в командном процессоре ОС UNIX».

Описаны упрощённый механизм семафоров, аналог команды `map` и генератор случайных букв через `$RANDOM`.

Сделаны выводы о достижении поставленной цели.

Содержание

1 Введение	7
1.1 Актуальность	7
1.2 Цель работы	7
1.3 Задачи	7
1.4 Объект и предмет исследования	8
1.5 Методы исследования	8
1.6 Схема выполнения работы	8
2 Теоретическая часть	10
2.1 Семафоры в UNIX	10
2.2 Сжатые man-страницы	10
2.3 Переменная \$RANDOM	10
3 Материалы и методы	11
4 Выполнение лабораторной работы	12
4.1 Задание 1. semaphore.sh	12
4.2 Задание 2. myman.sh — листинг	14
4.3 Задание 2. Запуск для ls	16
4.4 Задание 2. Запуск для grep	16
4.5 Задание 2. Сообщение об отсутствии справки	17
4.6 Задание 3. randletters.sh	17
5 Результаты	19
6 Обсуждение результатов	20
7 Заключение	21
Список литературы	22
А Приложение А. Ответы на контрольные вопросы	23
A.1 1. Синтаксическая ошибка в строке while [\$1 != "exit"]	23
A.2 2. Как объединить (конкатенация) несколько строк в одну?	23
A.3 3. Утилита seq и её альтернативы	24
A.4 4. Результат \$((10/3))	25
A.5 5. Основные отличия zsh от bash	26

A.6	6. Верен ли синтаксис <code>for ((a=1; a <= LIMIT; a++))?</code>	27
A.7	7. Сравнение <code>bash</code> с другими языками	28
В	Приложение Б. Листинги	30
B.1	Б.1. <code>semaphore.sh</code>	30
B.2	Б.2. <code>myman.sh</code>	31
B.3	Б.3. <code>randletters.sh</code>	31
С	Приложение В. Листинг команд	33

Список иллюстраций

1.1	Блок-схема выполнения лабораторной работы	9
4.1	Скрипт semaphore.sh	12
4.2	Запуск semaphore.sh	14
4.3	Скрипт myman.sh	15
4.4	./myman.sh ls	16
4.5	./myman.sh grep	16
4.6	./myman.sh nosuchcmd	17
4.7	Скрипт randletters.sh	17
4.8	Запуск randletters.sh	18

Список таблиц

5.1 Сводка результатов	19
----------------------------------	----

1 Введение

1.1 Актуальность

Командный процессор `bash` позволяет решать задачи синхронизации процессов, работы с архивами справок и генерации случайных данных. Эти навыки востребованы при администрировании UNIX-подобных систем и автоматизации рутинных операций.

1.2 Цель работы

Изучить основы программирования в оболочке ОС UNIX. Научиться писать более сложные командные файлы с использованием логических управляющих конструкций и циклов.

1.3 Задачи

1. Написать командный файл, реализующий упрощённый механизм семафоров.
2. Реализовать команду `map` с помощью командного файла.
3. Написать командный файл, генерирующий случайную последовательность букв через `$RANDOM`.
4. Ответить на контрольные вопросы.

1.4 Объект и предмет исследования

Объект — командная оболочка ОС UNIX.

Предмет — семафоры, работа с архивами tar, генерация случайных данных.

1.5 Методы исследования

Практическая работа в терминале, изучение tar-справок, написание скриптов, проверка результатов, фиксация скриншотами.

1.6 Схема выполнения работы

На рис. 1.1 представлена общая последовательность действий при выполнении работы.

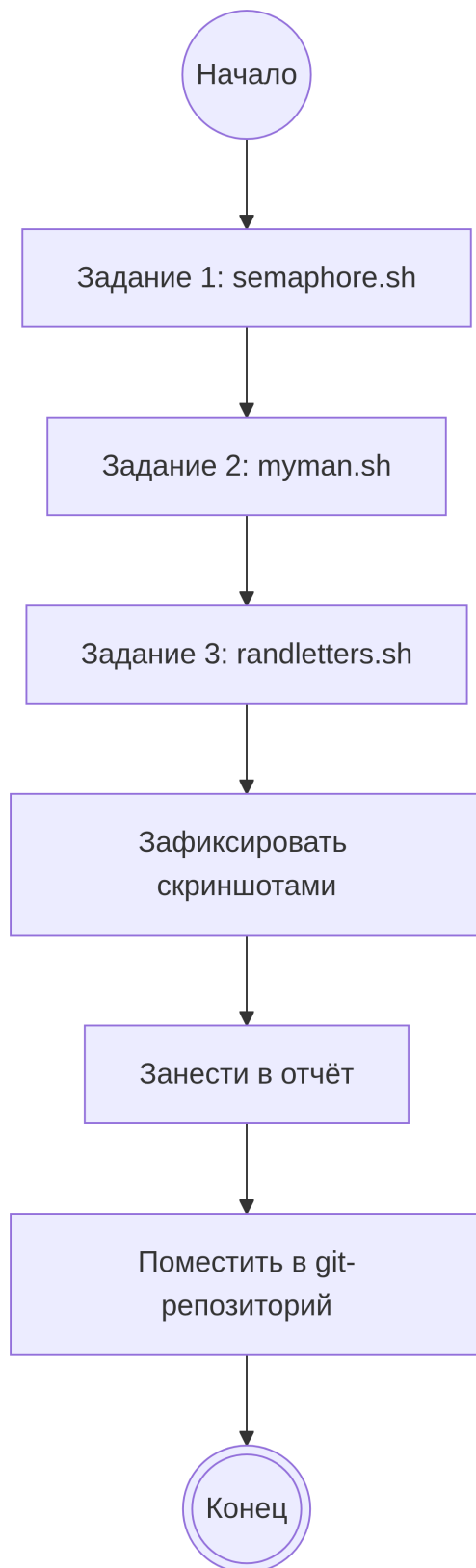


Рисунок 1.1: Блок-схема выполнения лабораторной работы

2 Теоретическая часть

2.1 Семафоры в UNIX

Семафор — механизм синхронизации процессов. В упрощённом виде реализуется через файл-маркер: если файл существует — ресурс занят, если нет — свободен.

2.2 Сжатые man-страницы

Man-страницы хранятся в `/usr/share/man/man1` как сжатые файлы `.1.gz`. Для просмотра: `zcat файл.1.gz | less` или `man -l файл.1.gz`.

2.3 Переменная `$RANDOM`

`$RANDOM` возвращает псевдослучайное число в диапазоне **0–32767**. Для получения буквы `a–z` используется остаток от деления на 26 и прибавление кода 97.

Более подробно про Unix см. в [1–4].

3 Материалы и методы

- **ОС:** Fedora 41 (Sway)
- **Терминал:** bash
- **Инструменты:** bash, zcat, less, groff, man, seq

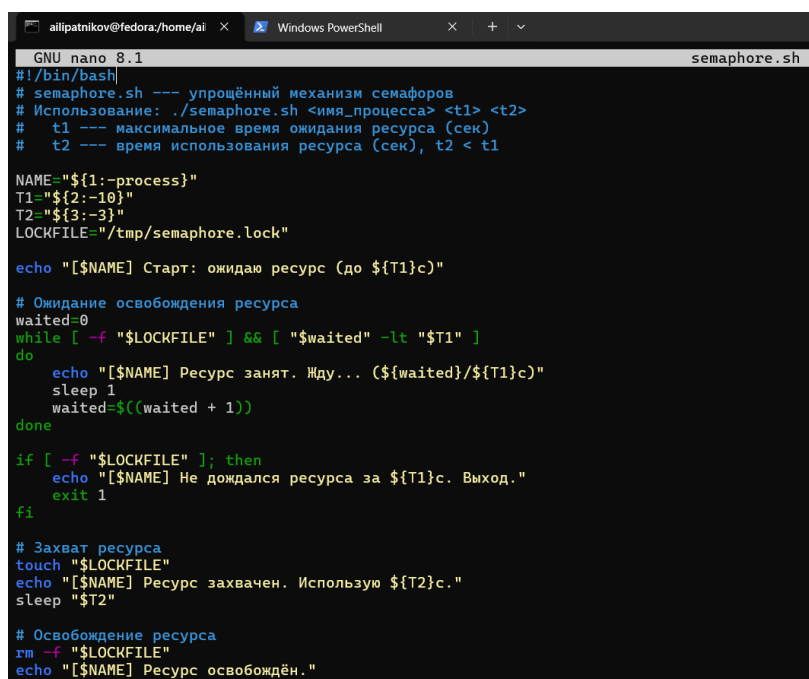
Рабочий каталог: `~/work/study/2025-2026/Операционные системы/os-intro/labs/lab14`.

Последовательность: написание трёх скриптов, проверка работы, фиксация **8 скриншотами**.

4 Выполнение лабораторной работы

4.1 Задание 1. semaphore.sh

Создан скрипт semaphore.sh (рис. 4.1).

The image shows a terminal window with the title 'GNU nano 8.1' and 'semaphore.sh'. The script content is as follows:

```
#!/bin/bash
# semaphore.sh --- упрощённый механизм семафоров
# Использование: ./semaphore.sh <имя_процесса> <t1> <t2>
#   t1 --- максимальное время ожидания ресурса (сек)
#   t2 --- время использования ресурса (сек), t2 < t1

NAME="${1:-process}"
T1="${2:-10}"
T2="${3:-3}"
LOCKFILE="/tmp/semaphore.lock"

echo "[$NAME] Старт: ожидаю ресурс (до ${T1}с)"

# Ожидание освобождения ресурса
waited=0
while [ -f "$LOCKFILE" ] && [ "$waited" -lt "$T1" ]
do
    echo "[$NAME] Ресурс занят. Жду... (${waited}/${T1}с)"
    sleep 1
    waited=$((waited + 1))
done

if [ -f "$LOCKFILE" ]; then
    echo "[$NAME] Не дождался ресурса за ${T1}с. Выход."
    exit 1
fi

# Захват ресурса
touch "$LOCKFILE"
echo "[$NAME] Ресурс захвачен. Использую ${T2}с."
sleep "$T2"

# Освобождение ресурса
rm -f "$LOCKFILE"
echo "[$NAME] Ресурс освобождён."
```

Рисунок 4.1: Скрипт semaphore.sh

```
#!/bin/bash
NAME="${1:-process}"
T1="${2:-10}"
T2="${3:-3}"
```

```

LOCKFILE="/tmp/semaphore.lock"

echo "[${NAME}] Старт: ожидаю ресурс (до ${T1}с)"

waited=0
while [ -f "$LOCKFILE" ] && [ "$waited" -lt "$T1" ]
do
    echo "[${NAME}] Ресурс занят. Жду... (${waited}/${T1}с)"
    sleep 1
    waited=$((waited + 1))
done

if [ -f "$LOCKFILE" ]; then
    echo "[${NAME}] Не дождался ресурса за ${T1}с. Выход."
    exit 1
fi

touch "$LOCKFILE"
echo "[${NAME}] Ресурс захвачен. Использую ${T2}с."
sleep "$T2"

rm -f "$LOCKFILE"
echo "[${NAME}] Освободил ресурс."

```

Запуск в двух терминалах (рис. 4.2).

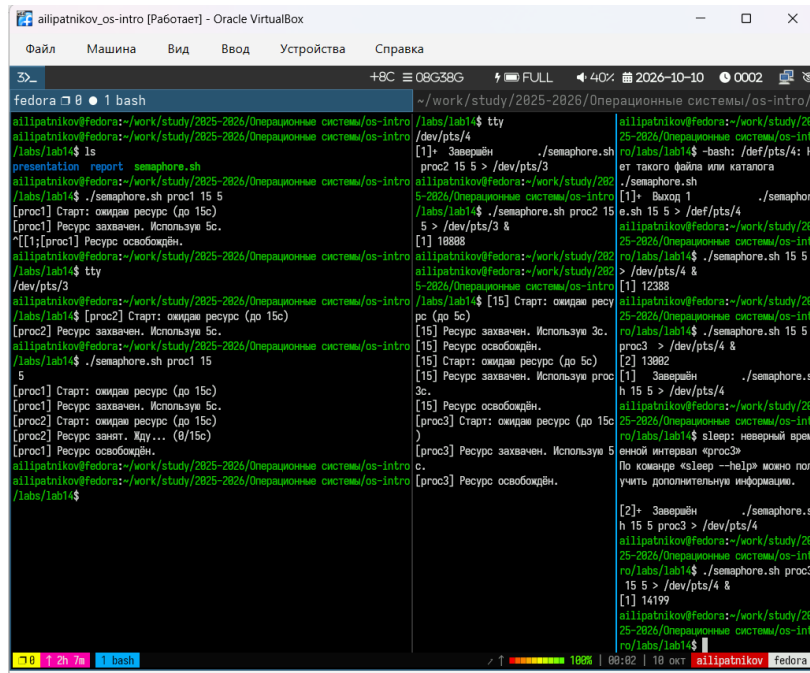


Рисунок 4.2: Запуск semaphore.sh

- Синхронизация через /tmp/semaphore.lock.
- Фоновый процесс пишет в /dev/pts/N.
- Поддерживается 3+ процессов.

4.2 Задание 2. myman.sh — листинг

Создан скрипт myman.sh (рис. 4.3).

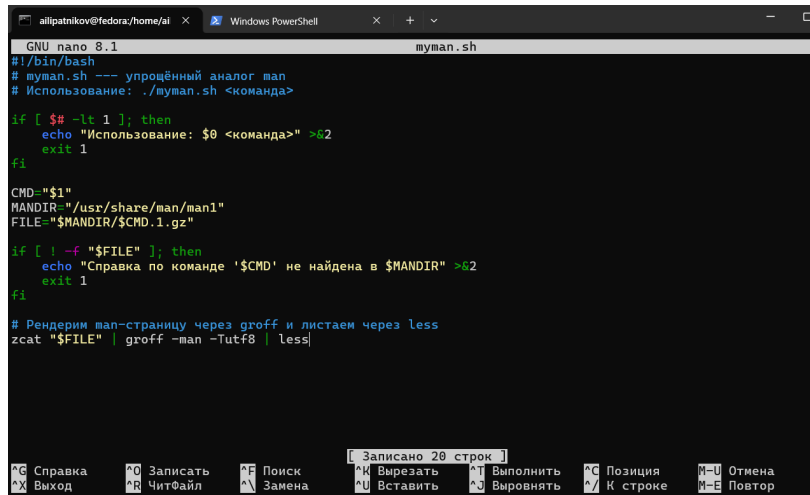


Рисунок 4.3: Скрипт myman.sh

```

#!/bin/bash

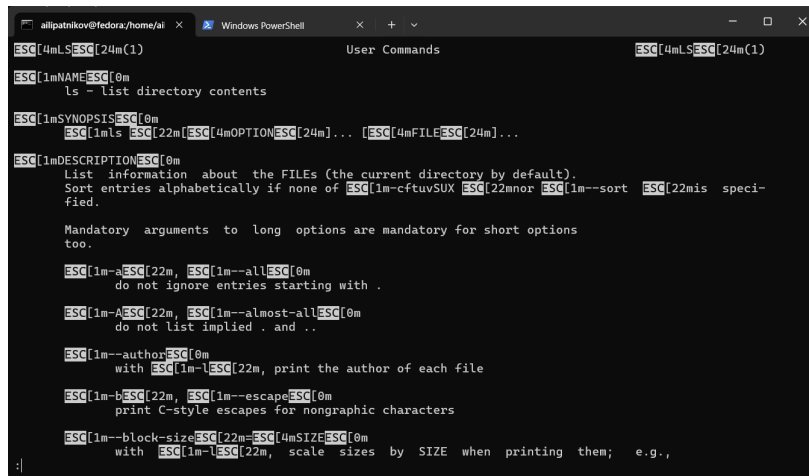
if [ $# -lt 1 ]; then
    echo "Использование: $0 <команда>" >&2
    exit 1
fi

CMD="$1"
MANDIR="/usr/share/man/man1"
FILE="$MANDIR/$CMD.1.gz"

if [ ! -f "$FILE" ]; then
    echo "Справка по команде '$CMD' не найдена в $MANDIR" >&2
    exit 1
fi

zcat "$FILE" | groff -man -Tutf8 | less
  
```

4.3 Задание 2. Запуск для ls

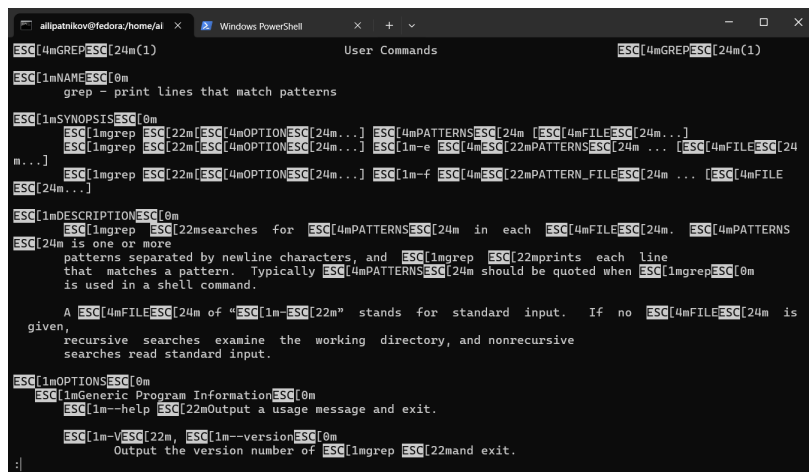


```
ESC[4mLSESC[24m(1) User Commands ESC[4mLSESC[24m(1)
ESC[1mNAMEESC[0m
ls - list directory contents
ESC[1mSYNOPSISESC[0m
ESC[1mls ESC[22m[ESC[4mOPTIONESC[24m]... [ESC[4mFILEESC[24m]...
ESC[1mDESCRIPTIONESC[0m
List information about the FILES (the current directory by default).
Sort entries alphabetically if none of ESC[1m-cftuvSUX ESC[22mnor ESC[1m--sort ESC[22mis speci-
fied.
Mandatory arguments to long options are mandatory for short options
too.
ESC[1m-aESC[22m, ESC[1m--allESC[0m
do not ignore entries starting with .
ESC[1m-AESC[22m, ESC[1m--almost-allESC[0m
do not list implied . and ..
ESC[1m--authorESC[0m
with ESC[1m-lESC[22m, print the author of each file
ESC[1m-bESC[22m, ESC[1m--escapeESC[0m
print C-style escapes for nongraphic characters
ESC[1m--block-sizeESC[22m=ESC[4mSIZEESC[0m
with ESC[1m-lESC[22m, scale sizes by SIZE when printing them; e.g.,
```

Рисунок 4.4: ./myman.sh ls

- Открывается справка по ls, отрендеренная через groff.
- Архив распаковывается через zcat.

4.4 Задание 2. Запуск для grep



```
ESC[4mGREPESC[24m(1) User Commands ESC[4mGREPESC[24m(1)
ESC[1mNAMEESC[0m
grep - print lines that match patterns
ESC[1mSYNOPSISESC[0m
ESC[1mgrep ESC[22m[ESC[4mOPTIONESC[24m]... ESC[4mPATTERNSESC[24m [ESC[4mFILEESC[24m]...
ESC[1mgrep ESC[22m[ESC[4mOPTIONESC[24m]... ESC[1m-e ESC[4mESC[22mPATTERNSESC[24m ... [ESC[4mFILEESC[24
m...]
ESC[1mgrep ESC[22m[ESC[4mOPTIONESC[24m]... ESC[1m-f ESC[4mESC[22mPATTERN_FILEESC[24m ... [ESC[4mFILE
ESC[24m...]
ESC[1mDESCRIPTIONESC[0m
ESC[1mgrep ESC[22msearches for ESC[4mPATTERNSESC[24m in each ESC[4mFILEESC[24m. ESC[4mPATTERNS
ESC[24m is one or more
patterns separated by newline characters, and ESC[1mgrep ESC[22mprints each line
that matches a pattern. Typically ESC[4mPATTERNSESC[24m should be quoted when ESC[1mgrepESC[0m
is used in a shell command.
A ESC[4mFILEESC[24m of "ESC[1m-ESC[22m" stands for standard input. If no ESC[4mFILEESC[24m is
given,
recursive searches examine the working directory, and nonrecursive
searches read standard input.
ESC[1mOPTIONSESC[0m
ESC[1mGeneric Program InformationESC[0m
ESC[1m--help ESC[22mOutput a usage message and exit.
ESC[1m-vESC[22m, ESC[1m--versionESC[0m
Output the version number of ESC[1mgrep ESC[22mand exit.
```

Рисунок 4.5: ./myman.sh grep

- Аналогично — справка по grep.

4.5 Задание 2. Сообщение об отсутствии справки

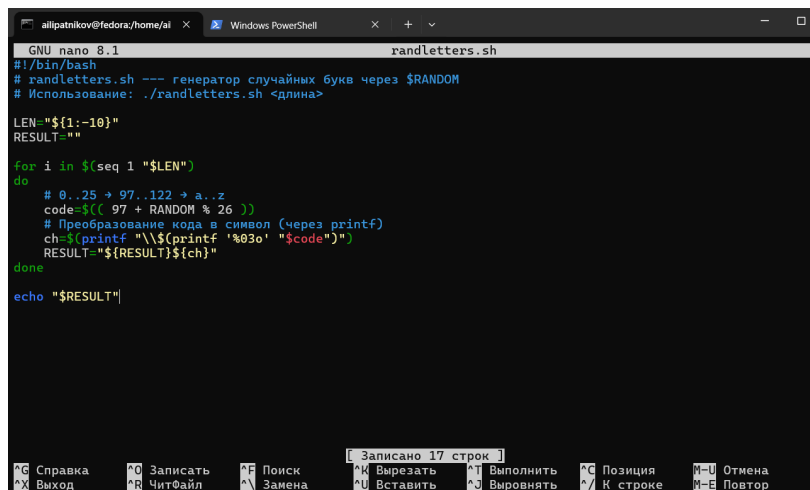
```
allipatnikov@fedora:~/work/study/2025-2026/Операционные системы/os-intro/labs/lab14$ nano myman.sh
allipatnikov@fedora:~/work/study/2025-2026/Операционные системы/os-intro/labs/lab14$ ./myman.sh ls
allipatnikov@fedora:~/work/study/2025-2026/Операционные системы/os-intro/labs/lab14$ ./myman.sh grep
allipatnikov@fedora:~/work/study/2025-2026/Операционные системы/os-intro/labs/lab14$ ./myman.sh nosuchcmd
Справка по команде 'nosuchcmd' не найдена в /usr/share/man/man1
allipatnikov@fedora:~/work/study/2025-2026/Операционные системы/os-intro/labs/lab14$ |
```

Рисунок 4.6: ./myman.sh nosuchcmd

- Если файла нет в /usr/share/man/man1 → сообщение об ошибке.
- Проверка [! -f «\$FILE»].

4.6 Задание 3. randletters.sh

Создан скрипт randletters.sh (рис. 4.7).



```
GNU nano 8.1 randletters.sh
#!/bin/bash
# randletters.sh --- генератор случайных букв через $RANDOM
# Использование: ./randletters.sh <длина>

LEN="${1:-10}"
RESULT=""

for i in $(seq 1 "$LEN")
do
    # 0..25 → 97..122 → a..z
    code=$(( 97 + RANDOM % 26 ))
    # Преобразование кода в символ (через printf)
    ch=$(printf "\\$(printf '%03o' "$code")")
    RESULT="${RESULT}${ch}"
done
echo "$RESULT"
```

Рисунок 4.7: Скрипт randletters.sh

```
#!/bin/bash

LEN="${1:-10}"
RESULT=""

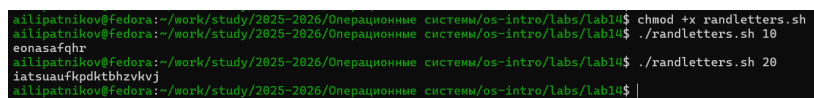
for i in $(seq 1 "$LEN")
do
```

```
code=$(( 97 + RANDOM % 26 ))
ch=$(printf "\\$(printf '%03o' "$code")")
RESULT="${RESULT}${ch}"
```

done

```
echo "$RESULT"
```

Запуск (рис. 4.8).



```
allipatnikov@fedora:~/work/study/2025-2026/Операционные системы/os-intro/labs/lab1$ chmod +x randletters.sh
allipatnikov@fedora:~/work/study/2025-2026/Операционные системы/os-intro/labs/lab1$ ./randletters.sh 10
eonasafqhr
allipatnikov@fedora:~/work/study/2025-2026/Операционные системы/os-intro/labs/lab1$ ./randletters.sh 20
iatsuaufkpdktbhzvkvj
allipatnikov@fedora:~/work/study/2025-2026/Операционные системы/os-intro/labs/lab1$ |
```

Рисунок 4.8: Запуск randletters.sh

- \$RANDOM % 26 → буква а–z.
- Каждый запуск даёт новую строку.
- Длина задаётся аргументом.

5 Результаты

Таблица 5.1: Сводка результатов

Задание	Файл	Рисунки	Результат
1	<code>semaphore.sh</code>	fig-001, fig-002	Синхронизация процессов
2	<code>myman.sh</code>	fig-003 — fig-006	Аналог <code>man</code>
3	<code>randletters.sh</code>	fig-007, fig-008	Случайные буквы

- 8 снимков: 2 на задание 1, 4 на задание 2, 2 на задание 3.
- Нумерация имён отражает содержимое.

6 Обсуждение результатов

Все три задания выполнены. Механизм семафоров через файл-маркер обеспечивает синхронизацию нескольких процессов: пока `/tmp/semaphore.lock` существует, другие процессы ждут. Скрипт `myman.sh` реализует упрощённый аналог `man`: ищет `.1.gz` в `/usr/share/man/man1`, распаковывает через `zcat`, рендерит через `groff` и листает через `less`; при отсутствии файла выдаёт сообщение. Генератор `randletters.sh` использует `$RANDOM` для получения букв `a-z`.

7 Заключение

- Цель работы достигнута: изучены основы программирования в оболочке UNIX.
- Освоен механизм семафоров через файл-маркер.
- Реализован упрощённый аналог map.
- Освоена генерация случайных данных через \$RANDOM.

Список литературы

1. *Newham C.* Learning the bash Shell: Unix Shell Programming. — O'Reilly Media, 2005. — 354 p. — (In a Nutshell). — ISBN 0596009658. — URL: <http://www.amazon.com/Learning-bash-Shell-Programming-Nutshell/dp/0596009658>.
2. *Robbins A.* Bash Pocket Reference. — O'Reilly Media, 2016. — 156 p. — ISBN 978-1491941591.
3. *Zarrelli G.* Mastering Bash. — Packt Publishing, 2017. — 502 p. — ISBN 9781784396879.
4. *Таненбаум Э., Бос Х.* Современные операционные системы.. — 4-е изд. — СПб.: Питер, 2015.. — 1120 с. — (Классика Computer Science).

А Приложение А. Ответы на контрольные вопросы

А.1 1. Синтаксическая ошибка в строке `while [$1 != "exit"]`

Ошибка: отсутствуют **пробелы** после `[` и перед `]`.

Правильно:

```
while [ "$1" != "exit" ]
```

Почему: `[` — это **команда** (синоним `test`). Как и любая команда, она требует **пробелов** между именем команды и аргументами. Без пробелов `bash` пытается выполнить команду с именем `[$1`, которой не существует.

Дополнительно: переменную `$1` рекомендуется **брать в кавычки** — на случай пробелов в значении.

А.2 2. Как объединить (конкатенация) несколько строк в одну?

Способы:

1. Присваивание с подстановкой переменных:

```
line1="Hello, "  
line2="world!"  
combined="${line1}${line2}"  
echo "$combined"      # Hello, world!
```

2. Через += (добавление к переменной):

```
result=""  
result+="Hello, "  
result+="world!"  
echo "$result"
```

3. Объединение файлов:

```
cat file1.txt file2.txt > combined.txt
```

4. Многострочная переменная:

```
text="строка1  
строка2  
строка3"
```

5. Через printf:

```
combined=$(printf "%s%s" "$line1" "$line2")
```

А.3 3. Утилита seq и её альтернативы

seq — генерирует последовательность чисел:

```
seq 1 5          # 1 2 3 4 5
seq 1 2 10       # 1 3 5 7 9 (шаг 2)
seq -w 1 10      # 01 02 03 ... 10 (с ведущими нулями)
```

Альтернативы в bash:

1. Фигурные скобки (brace expansion):

```
for i in {1..10}; do echo "$i"; done
```

2. С-подобный цикл:

```
for ((i=1; i<=10; i++)); do echo "$i"; done
```

3. while со счётчиком:

```
i=1
while [ $i -le 10 ]; do echo "$i"; i=$((i+1)); done
```

4. printf в цикле:

```
printf "%d\n" {1..10}
```

5. Через awk:

```
awk 'BEGIN { for(i=1; i<=10; i++) print i }'
```

Вывод: seq — не единственный способ; встроенные конструкции bash ({1..10}, for (...)) работают быстрее и без внешних утилит.

А.4 4. Результат $\$(10/3)$

Результат: 3.

`$((...))` выполняет **целочисленную арифметику**. Дробная часть **отбрасывается**.

$10 / 3 = 3.333... \rightarrow 3$.

Для вещественного деления нужны `bc`, `awk` или `zsh`:

```
echo "scale=2; 10/3" | bc      # 3.33
awk 'BEGIN { print 10/3 }'     # 3.33333
```

А.5 5. Основные отличия `zsh` от `bash`

Признак	bash	zsh
Расширение глоббинга	Базовое (*, ?, [...])	Расширенное (**/*, ^, <...>, рекурсивный **)
Автодополнение	Базовое	Продвинутое (контекстное, с меню)
Коррекция опечаток	Нет	<code>setopt correct</code>
Массивы	С 1D индексами	С 1-based и ассоциативными
Плагины	Ограничены	Oh My Zsh, Prezto и др.
Промпт	Простой (PS1)	Тематизированный, с %-расширениями
Скорость	Быстрее	Немного медленнее
Совместимость с POSIX	Полная	Частичная

Признак	bash	zsh
Популярность в скриптах	Стандарт	Реже (обычно для интерактивной работы)

Вывод: bash — стандарт для скриптов (POSIX-совместимость, установлен везде). zsh — удобнее для **интерактивной** работы.

А.6 6. Верен ли синтаксис `for ((a=1; a <= LIMIT; a++))`?

Да, синтаксис верен — это **арифметический цикл** bash (C-подобный).

Условия корректности: 1. Переменная `LIMIT` должна быть определена — иначе цикл не выполнится (сравнение с пустой строкой → ошибка или ложь). 2.

Внутри ((...)) — пробелы **допустимы** (в отличие от `[ ]`). 3. **Двойные круглые скобки** — обязательны для арифметики.

Пример:

```
LIMIT=5
for ((a=1; a <= LIMIT; a++))
do
    echo "a = $a"
done
```

Вывод: `a = 1, a = 2, ..., a = 5.`

Ошибка возникнет, если: - `LIMIT` не задана. - Пропущена `;` или перенос строки перед `do`.

A.7 7. Сравнение bash с другими языками

A.7.1 Преимущества bash

Преимущество	Пояснение
Нативная работа с ОС	Прямой доступ к файлам, процессам, сигналам
Склеивание утилит	Конвейеры <code>\ </code> , перенаправления
Не требует компиляции	Интерпретируется на ходу
Установлен везде	POSIX-совместимая оболочка
Быстрое прототипирование	Однострочники для задач автоматизации
Встроенные glob-паттерны	<code>*</code> , <code>?</code> , <code>[a-z]</code>

A.7.2 Недостатки bash

Недостаток	Пояснение
Медленный	Интерпретация + запуск внешних процессов на каждую операцию
Слабая типизация	Всё — строка (кроме арифметики)
Нет сложных структур данных	Массивы 1D, нет хэшей (в bash — только ассоциативные с 4.0)
Плохая обработка ошибок	<code>set -e</code> — грубый инструмент
Ограниченная отладка	<code>bash -x</code> — не всегда удобно
Не подходит для алгоритмов	Рекурсия, сортировки, математика — плохо

А.7.3 Сравнение с конкретными языками

bash vs Python: - bash — быстро для shell-задач (файлы, процессы). - Python — быстрее для алгоритмов, лучше структуры данных.

bash vs Perl: - Perl мощнее в **regex**. - bash проще для **оркестрации** утилит.

bash vs C: - C быстрее, но требует **компиляции**. - bash — интерпретация «на лету».

Вывод: bash — **не универсальный** язык, а **клей** между утилитами UNIX. Для сложной логики — лучше Python/Perl. Для быстрых операций с файлами и процессами — bash незаменим.

В Приложение Б. Листинги

В.1 Б.1. semaphore.sh

```
#!/bin/bash
NAME="${1:-process}"
T1="${2:-10}"
T2="${3:-3}"
LOCKFILE="/tmp/semaphore.lock"
echo "[${NAME}] Старт: ожидаю ресурс (до ${T1}с)"
waited=0
while [ -f "$LOCKFILE" ] && [ "$waited" -lt "$T1" ]
do
    echo "[${NAME}] Ресурс занят. Жду... (${waited}/${T1}с)"
    sleep 1
    waited=$((waited + 1))
done
if [ -f "$LOCKFILE" ]; then
    echo "[${NAME}] Не дождался ресурса за ${T1}с. Выход."
    exit 1
fi
touch "$LOCKFILE"
echo "[${NAME}] Ресурс захвачен. Использую ${T2}с."
```

```
sleep "$T2"
rm -f "$LOCKFILE"
echo "[${NAME}] Освободил ресурс."
```

B.2 Б.2. myman.sh

```
#!/bin/bash
if [ $# -lt 1 ]; then
    echo "Использование: $0 <команда>" >&2
    exit 1
fi
CMD="$1"
MANDIR="/usr/share/man/man1"
FILE="$MANDIR/$CMD.1.gz"
if [ ! -f "$FILE" ]; then
    echo "Справка по команде '$CMD' не найдена в $MANDIR" >&2
    exit 1
fi
zcat "$FILE" | groff -man -Tutf8 | less
```

B.3 Б.3. randletters.sh

```
#!/bin/bash
LEN="${1:-10}"
RESULT=""
for i in $(seq 1 "$LEN")
```

do

```
code=$(( 97 + RANDOM % 26 ))
```

```
ch=$(printf "\\$(printf '%03o' "$code")")
```

```
RESULT="${RESULT}${ch}"
```

done

```
echo "$RESULT"
```

С Приложение В. Листинг команд

```
cd ~/work/study/2025-2026/"Операционные системы"/os-intro/labs/lab14

# Задание 1
nano semaphore.sh
chmod +x semaphore.sh
./semaphore.sh proc1 15 5
# в другом терминале
./semaphore.sh proc2 15 5 > /dev/pts/1 &

# Задание 2
nano myman.sh
chmod +x myman.sh
./myman.sh ls
./myman.sh grep
./myman.sh nosuchcmd

# Задание 3
nano randletters.sh
chmod +x randletters.sh
./randletters.sh 10
./randletters.sh 20
```